Package 'concurve'

June 15, 2026

Type Package

Title Computes & Plots Compatibility (Confidence), Surprisal, & Likelihood Distributions

Version 3.0.0

Date 2026-01-26

Maintainer Zad Rafi <zad@lesslikely.com>

Description
Computes compatibility (confidence) distributions along with their corresponding P-values, S-values, and likelihoods. The intervals can be plotted to form the distributions themselves. Functions can be compared to one another to see how much they overlap. Results can be exported to Microsoft Word, Powerpoint, and TeX documents. The package currently supports resampling methods, computing differences, generalized linear models, mixed-effects models, survival analysis, and meta-analysis. These methods are discussed by Schweder T, Hjort NL. (2016, ISBN:9781316445051) and Rafi Z, Greenland S. (2020) <doi:10.1186/s12874-020-01105-9>.

License GPL-3 | file LICENSE

URL https://stat.lesslikely.com/concurve/, https://github.com/zadrafi/concurve

BugReports https://github.com/zadrafi/concurve/issues

Depends R (>= 4.0.0)

Imports methods, bcaboot, boot, dplyr, flextable, ggplot2, knitr, metafor, officer, parallel, ProfileLikelihood, scales, colorspace, survival, survminer, tibble,

tidyr,
lme4,
pbmcapply

Suggests MASS,
covr,
roxygen2,
spelling,
testthat,
rmarkdown,
Lock5Data,
carData,
bench,
rms,
brms,
rstan,
rstanarm,
bayesplot,
vdiffr,
ggtext,
daewr,
svglite,
data.table,
nlme,
simstudy,
patchwork,
cowplot,
reprex,
roxygen2md,
kableExtra,
magick,
magrittr,
Statamarkdown,
numDeriv,
jsonlite,
openxlsx,
plotly,
DBI,
odbc

VignetteBuilder knitr

ByteCompile true

Encoding UTF-8

Language en-US

LazyData true

Roxygen list(markdown = TRUE)

X-schema.org-keywords confidence, compatibility, consonance, curve,
information statistics, surprisals, interval, function, distribution, fiducial

Config/roxygen2/version 8.0.0

Contents

coef.likelihood_function . . . 3
confint.likelihood_function . . . 4
construct_likelihood . . . 5
curve_boot . . . 6
curve_compare . . . 7
curve_corr . . . 8
curve_from_ratio . . . 9
curve_from_se . . . 10
curve_gen . . . 11
curve_lik . . . 13
curve_lmer . . . 13
curve_mean . . . 15
curve_meta . . . 16
curve_model . . . 18
curve_overlap . . . 20
curve_rev . . . 21
curve_snowflake . . . 22
curve_snowflake_batch . . . 24
curve_summary . . . 25
curve_surv . . . 26
curve_table . . . 27
curve_wrap . . . 28
export_for_powerbi . . . 30
ggcurve . . . 32
ggplot_likelihood . . . 34
logLik.likelihood_function . . . 35
plot.likelihood_function . . . 35
plotly_likelihood . . . 37
plot_all_parameters . . . 38
plot_ci_levels . . . 38
plot_compare . . . 39
plot_multi . . . 42
plot_profile_vs_wald . . . 44
print.likelihood_function . . . 45
print.summary.likelihood_function . . . 45
RobustMax . . . 46
RobustMin . . . 46
summary.likelihood_function . . . 47
vcov.likelihood_function . . . 47

Index **49**

`coef.likelihood_function` *Extract coefficients from a likelihood function*

Description

Extract coefficients from a likelihood function

Usage

```
## S3 method for class 'likelihood_function'
coef(object, ...)
```

Arguments

object	A likelihood_function object
...	Additional arguments (unused)

Details

Returns the parameter estimates (MLE) from the likelihood function.

Value

A named numeric vector of maximum likelihood estimates

---

confint.likelihood_function

Confidence intervals for likelihood function parameters

Description

Confidence intervals for likelihood function parameters

Usage

```
## S3 method for class 'likelihood_function'
confint(object, parm = NULL, level = 0.95, ...)
```

Arguments

object	A likelihood_function object
parm	A character vector of parameter names for which to compute intervals. If NULL, intervals are computed for all parameters.
level	Confidence level (default: 0.95)
...	Additional arguments (unused)

Details

Uses profile likelihood methodology to construct confidence intervals. The intervals are obtained by finding where the profile log-likelihood drops by $\chi^2_{1,\alpha}/2$ from its maximum.

Value

A matrix with two columns containing the lower and upper confidence limits for each parameter. Column names indicate the confidence level.

`construct_likelihood` *Construct Likelihood Function for Statistical Models*

Description

Build likelihood, log-likelihood, and deviance functions from scratch without external dependencies. Supports profile likelihood and likelihood-based inference.

Usage

```
construct_likelihood(
  model = NULL,
  data = NULL,
  formula = NULL,
  family = gaussian(),
  method = c("auto", "numeric", "analytic")
)
```

Arguments

`model`	Fitted model object (lm, glm, etc.) or NULL to build from scratch
`data`	Data frame (required if model is NULL)
`formula`	Model formula (required if model is NULL)
`family`	Family object for GLMs (default: gaussian)
`method`	Method for likelihood construction: "auto", "numeric", "analytic"

Details

Based on:

- Pawitan (2001). In All Likelihood. Oxford University Press.
- Venzon & Moolgavkar (1988). A method for computing profile-likelihood-based confidence intervals. Applied Statistics, 37(1), 87-94.
- Murphy & van der Vaart (2000). On profile likelihood. JASA, 95(450), 449-465.

Value

List containing likelihood functions and methods

curve_boot *Generate Consonance Functions via Bootstrapping*

Description

Use the Bca bootstrap method and the t-boostrap method from the bcaboot and boot packages to generate consonance distrbutions.

Usage

```
curve_boot(
  data = data,
  func = func,
  method = "bca",
  t0,
  tt,
  bb,
  replicates = 2000,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

data	Dataset that is being used to create a consonance function.
func	Custom function that is used to create parameters of interest that will be bootstrapped.
method	The boostrap method that will be used to generate the functions. Methods include "bca" which is the default, "bcapar", which is parametric bootstrapping using the bca method and "t", for the t-bootstrap/percentile method.
t0	Only used for the "bcapar" method. Observed estimate of theta, usually by maximum likelihood.
tt	Only used for the "bcapar" method. A vector of parametric bootstrap replications of theta of length B, usually large, say B = 2000
bb	Only used for the "bcapar" method. A B by p matrix of natural sufficient vectors, where p is the dimension of the exponential family.
replicates	Indicates how many bootstrap replicates are to be performed. The default is currently 20000 but more may be desirable, especially to make the functions more smooth.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 7 items where the dataframe of standard values is in the first list and the table for it in the second if table = TRUE. The Bca intervals and table are found in the third and fourth list. The values for the density function are in the fifth object, while the Bca stats are in the sixth and seventh objects.

`curve_compare` *Compare Two Functions and Produces An AUC Score*

Description

Compares the p-value/s-value, and likelihood functions and computes an AUC number.

Usage

```
curve_compare(data1, data2, type = "c", plot = TRUE, ...)
```

Arguments

`data1`	The first dataframe produced by one of the interval functions in which the intervals are stored.
`data2`	The second dataframe produced by one of the interval functions in which the intervals are stored.
`type`	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example curve_compare (type = "s") or curve_compare(type = "c"). Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
`plot`	by default it is set to TRUE and will use the plot_compare() function to plot the two functions.
`...`	Can be used to pass further arguments to plot_compare().

Value

Computes an AUC score and returns a plot that graphs two functions.

See Also

`plot_compare()`

`ggcurve()`

`curve_table()`

Examples

```
## Not run:
library(concurve)
GroupA <- rnorm(50)
GroupB <- rnorm(50)
RandomData <- data.frame(GroupA, GroupB)
intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData)
GroupA2 <- rnorm(50)
GroupB2 <- rnorm(50)
RandomData2 <- data.frame(GroupA2, GroupB2)
model <- lm(GroupA2 ~ GroupB2, data = RandomData2)
randomframe <- curve_gen(model, "GroupB2")
curve_compare(intervalsdf[[1]], randomframe[[1]])

## End(Not run)
```

---

curve_corr *Consonance Functions for Correlations*

Description

Computes consonance intervals to produce P- and S-value functions for correlational analysesusing the cor.test function in base R and places the interval limits for each interval levelinto a data frame along with the corresponding p-values and s-values.

Usage

```
curve_corr(
  x,
  y,
  alternative,
  method,
  steps = 10000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

x	A vector that contains the data for one of the variables that will be analyzed for correlational analysis.
y	A vector that contains the data for one of the variables that will be analyzed for correlational analysis.
alternative	Indicates the alternative hypothesis and must be one of "two.sided", "greater" or "less". You can specify just the initial letter. "greater" corresponds to positive association, "less" to negative association.
method	A character string indicating which correlation coefficient is to be used for the test. One of "pearson", "kendall", or "spearman", can be abbreviated.

steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
GroupA <- rnorm(50)
GroupB <- rnorm(50)
joe <- curve_corr(x = GroupA, y = GroupB, alternative = "two.sided", method = "pearson")

## End(Not run)
```

---

curve_from_ratio *Construct Consonance Function from Ratio Estimate*

Description

Convenience function to construct consonance functions from ratio measures (odds ratios, hazard ratios, risk ratios) given a point estimate and confidence interval bounds.

Usage

```
curve_from_ratio(
  ratio,
  lower,
  upper,
  conf.level = 0.95,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

ratio	Point estimate of the ratio.
lower	Lower bound of the confidence interval.
upper	Upper bound of the confidence interval.
conf.level	Confidence level of the provided interval. Default is 0.95.

steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This is a convenience wrapper around curve_rev specifically for ratio measures. It automatically handles the log transformation and sets appropriate measure type.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

curve_from_se() for constructing from standard error

curve_rev() for the underlying function

Examples

```
## Not run:
# From a published odds ratio: OR = 1.5, 95% CI [1.1, 2.0]
result <- curve_from_ratio(ratio = 1.5, lower = 1.1, upper = 2.0)
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

# Hazard ratio from survival analysis: HR = 0.75, 95% CI [0.60, 0.95]
result <- curve_from_ratio(ratio = 0.75, lower = 0.60, upper = 0.95)
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

## End(Not run)
```

---

curve_from_se *Construct Consonance Function from Standard Error*

Description

Convenience function to construct consonance functions given a point estimate and its standard error.

Usage

```
curve_from_se(
  estimate,
  se,
  measure = "mean",
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

estimate	Point estimate.
se	Standard error of the estimate.
measure	Type of measure: "mean" for differences (default), "ratio" for ratio measures (estimate should be on original scale).
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This function constructs consonance intervals using the normal approximation: estimate +/- z * se where z varies across confidence levels.

For ratio measures, the function handles the log transformation internally.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

curve_from_ratio() for constructing from CI bounds

curve_rev() for the underlying function

Examples

```
## Not run:
# Mean difference with SE
result <- curve_from_se(estimate = 2.5, se = 0.8, measure = "mean")
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

# Odds ratio with SE (on log scale internally)
result <- curve_from_se(estimate = 1.5, se = 0.2, measure = "ratio")
ggcurve(result[[1]], type = "c", nullvalue = TRUE)

## End(Not run)
```

---

curve_gen *Consonance Functions For Linear Models, Generalized Linear Models, and Robust Linear Models*

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the selected model (linear models, general linear models, robust linear models, and generalized least squares and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values. Can also adjust for multiple comparisons. It is generally recommended to wrap this function using suppressMessages() due to the long list of profiling messages.

Arguments

model	The statistical model of interest (ANOVA, regression, logistic regression) is to be indicated here.
var	The variable of interest from the model (coefficients, intercept) for which the intervals are to be produced.
method	Chooses the method to be used to calculate the consonance intervals. There are currently five methods: "lm", rms::ols objects can be used with the "lm" option, "rlm", "glm" and "aov", and "gls". The "lm" method uses the profile likelihood method to compute intervals and can be used for models created by the 'lm' function. It is typically what most people are familiar with when computing intervals based on the calculated standard error. The ols function from the rms package can also be used for this option. The "rlm" method is designed for usage with the "rlm" function from the MASS package. The "glm" method allows this function to be used for specific scenarios like logistic regression and the 'glm' function. Similarly, the Glm function from the rms package can also be used for this option. The gls method allows objects from gls() or from Gls() from the rms package.
log	Determines whether the coefficients will be exponentiated or not. By default, it is off and set to FALSE or F, but changing this to TRUE or T, will exponentiate the results which may be useful if trying to view the results from a logistic regression on a scale that is not logarithmic.
penalty	An input to specify whether the confidence intervals should be corrected for multiple comparisons. The default is NULL, so there is no correction. Other options include "bonferroni" and "sidak".
m	Indicates how many comparisons are being done and the number that should be used to correct for multiple comparisons. The default is NULL.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a data frame.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
# Simulate random data
GroupA <- rnorm(50)
GroupB <- rnorm(50)
RandomData <- data.frame(GroupA, GroupB)
rob <- lm(GroupA ~ GroupB, data = RandomData)
bob <- curve_gen(rob, "GroupB")
```

```
## End(Not run)
```

---

curve_lik *Compute Profile Likelihood Functions*

Description

Compute Profile Likelihood Functions

Usage

```
curve_lik(likobject, data, table = TRUE)
```

Arguments

likobject	An object from the ProfileLikelihood package
data	The dataframe that was used to create the likelihood object in the ProfileLikelihood package.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 2 items where the dataframe of values is in the first object, and the table for the values in the second if table = TRUE.

Examples

```
library(ProfileLikelihood)
data(dataglm)
xx <- profilelike.glm(y ~ x1 + x2, dataglm, profile.theta = "group", binomial("logit"))
lik <- curve_lik(xx, dataglm)
```

---

curve_lmer *Consonance Functions For Linear & Non-Linear Mixed-Effects Models.*

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the selected lme4 model and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.. It is generally recommended to wrap this function using suppressMessages() due to the long list of profiling messages

Usage

```
curve_lmer(
  object,
  parm,
  method = "profile",
  zeta = NULL,
  nsim = NULL,
  FUN = NULL,
  boot.type = NULL,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = FALSE
)
```

Arguments

object	The statistical model of interest from lme4 is to be indicated here.
parm	The variable of interest from the model (coefficients, intercept) for which the intervals are to be produced.
method	Chooses the method to be used to calculate the consonance intervals. There are currently four methods: "default", "wald", "lm", and "boot". The "default" method uses the profile likelihood method to compute intervals and can be used for models created by the 'lm' function. The "wald" method is typicallywhat most people are familiar with when computing intervals based on the calculated standard error. The "lm" method allows this function to be used for specific scenarios like logistic regression and the 'glm' function. The "boot" method allows for bootstrapping at certain levels.
zeta	(for method = "profile" only:) likelihood cutoff (if not specified, as by default, computed from level).
nsim	number of simulations for parametric bootstrap intervals.
FUN	function; if NULL, an internal function that returns the fixed-effect parameters as well as the random-effect parameters on the standard deviation/correlationscale will be used.
boot.type	bootstrap confidence interval type, as described in boot.c i. Methods stud and bca are unavailable because they require additional components to be calculated.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

curve_mean *Consonance Functions For Mean Differences*

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in a statistical test that compares means and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.

Usage

```
curve_mean(
  x,
  y,
  data,
  paired = F,
  method = "default",
  replicates = 1000,
  steps = 10000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

x	Variable that contains the data for the first group being compared.
y	Variable that contains the data for the second group being compared.
data	Data frame from which the variables are being extracted from.
paired	Indicates whether the statistical test is a paired difference test. By default, it is set to "F",which means the function will be an unpaired statistical test comparing two independent groups.Inserting "paired" will change the test to a paired difference test.
method	By default this is turned off (set to "default"), but allows for bootstrapping if "boot" is inserted into the function call.
replicates	Indicates how many bootstrap replicates are to be performed. The default is currently 20000 but more may be desirable, especially to make the functions more smooth.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
# Simulate random data
GroupA <- runif(100, min = 0, max = 100)
GroupB <- runif(100, min = 0, max = 100)
RandomData <- data.frame(GroupA, GroupB)
bob <- curve_mean(GroupA, GroupB, RandomData)

## End(Not run)
```

`curve_meta` *Consonance Functions For Meta-Analytic Data*

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the meta-analysis done by the metafor package and places the interval limits for each interval level into a data frame along with the corresponding p-values and s-values.

Usage

```
curve_meta(
  x,
  measure = "default",
  method = "uni",
  parm = NULL,
  robust = FALSE,
  cluster = NULL,
  adjust = FALSE,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

`x`	Object where the meta-analysis parameters are stored, typically a list produced by 'metafor'
`measure`	Indicates whether the object has a log transformation or is normal/default. The default setting is "default. If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes such as risk ratios, hazard ratios, and odds ratios.
`method`	Indicates which meta-analysis metafor function is being used. Currently supports rma.uni ("uni"), which is the default, rma.mh ("mh"), and rma.peto ("peto")

parm	Typically ignored, but needed sometimes in order to specify which variable to produce function for.
robust	a logical indicating whether to produce cluster robust interval estimates Default is FALSE.
cluster	a vector specifying a clustering variable to use for constructing the sandwich estimator of the variance-covariance matrix. Default setting is NULL.
adjust	logical indicating whether a small-sample correction should be applied to the variance-covariance matrix. Default is FALSE.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
# Simulate random data for two groups in two studies
GroupAData <- runif(20, min = 0, max = 100)
GroupAMean <- round(mean(GroupAData), digits = 2)
GroupASD <- round(sd(GroupAData), digits = 2)

GroupBData <- runif(20, min = 0, max = 100)
GroupBMean <- round(mean(GroupBData), digits = 2)
GroupBSD <- round(sd(GroupBData), digits = 2)

GroupCData <- runif(20, min = 0, max = 100)
GroupCMean <- round(mean(GroupCData), digits = 2)
GroupCSD <- round(sd(GroupCData), digits = 2)

GroupDData <- runif(20, min = 0, max = 100)
GroupDMean <- round(mean(GroupDData), digits = 2)
GroupDSD <- round(sd(GroupDData), digits = 2)

# Combine the data

StudyName <- c("Study1", "Study2")
MeanTreatment <- c(GroupAMean, GroupCMean)
MeanControl <- c(GroupBMean, GroupDMean)
SDTreatment <- c(GroupASD, GroupCSD)
SDControl <- c(GroupBSD, GroupDSD)
NTreatment <- c(20, 20)
NControl <- c(20, 20)
```

```
metadf <- data.frame(
  StudyName, MeanTreatment, MeanControl,
  SDTreatment, SDControl, NTreatment, NControl
)

# Use metafor to calculate the standardized mean difference

library(metafor)

dat <- escalc(
  measure = "SMD", m1i = MeanTreatment, sd1i = SDTreatment,
  n1i = NTreatment, m2i = MeanControl, sd2i = SDControl,
  n2i = NControl, data = metadf
)

# Pool the data using a particular method. Here "FE" is the fixed-effects model

res <- rma(yi, vi,
  data = dat, slab = paste(StudyName, sep = ", "),
  method = "FE", digits = 2
)

# Calculate the intervals using the metainterval function

metaf <- curve_meta(res)

## End(Not run)
```

`curve_model` *Construct Consonance Functions from Fitted Models*

Description

A convenience wrapper around `curve_wrap` for common model objects. Automatically selects the appropriate confidence interval method based on the model class or user specification.

Usage

```
curve_model(
  model,
  param,
  method = "default",
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

`model`	A fitted model object (lm, glm, nls, lme, etc.)
`param`	Character string specifying which parameter to extract.
`method`	Method for computing confidence intervals:

	"default" Uses `confint.default()` - Wald/normal approximation (fast) **"profile"** Uses `confint()` - profile likelihood (slower, more accurate for GLMs)
`steps`	Number of consonance levels to compute. Default is 1000.
`cores`	Number of cores for parallel computation.
`table`	Logical. If TRUE (default), includes a summary table.

Details

This is a convenience function that constructs the appropriate `ci_func` for `curve_wrap` based on the specified method. For maximum flexibility, use `curve_wrap` directly.

Method selection guidelines:

- **Linear models (lm)**: "default" is appropriate and fast
- **GLMs**: "profile" is more accurate, especially for small samples
- **Mixed models**: "profile" recommended but can be slow
- **Robust models (rlm)**: "default" typically used

Value

A list with class "concurve" (see `curve_wrap` for details).

See Also

`curve_wrap()` for the generic wrapper

`curve_gen()` for the original model-based function

`ggcurve()` for plotting

Examples

```
## Not run:
# Linear model - Wald intervals (fast)
model <- lm(mpg ~ wt + hp, data = mtcars)
result <- curve_model(model, param = "wt", method = "default")
ggcurve(result[[1]], type = "c")

# GLM - Profile likelihood intervals (more accurate)
model <- glm(am ~ wt, data = mtcars, family = binomial)
result <- curve_model(model, param = "wt", method = "profile")

# Compare methods
wald <- curve_model(model, param = "wt", method = "default")
profile <- curve_model(model, param = "wt", method = "profile")
# Plot both to see the difference

## End(Not run)
```

curve_overlap *Calculate Overlap Between Consonance Functions*

Description

Quantifies the area of overlap between two consonance functions, providing a measure of compatibility between two estimates.

Usage

```
curve_overlap(
  data1,
  data2,
  type = "c",
  plot = TRUE,
  title = "Consonance Function Overlap"
)
```

Arguments

data1	First concurve object or intervals data frame.
data2	Second concurve object or intervals data frame.
type	Function type for comparison: "c" for consonance (default), "s" for surprisal.
plot	Logical. If TRUE (default), displays overlap visualization.
title	Plot title.

Details

This function computes the overlap between two consonance functions, which provides a measure of how compatible two estimates are with each other. Higher overlap indicates greater compatibility.

The overlap is calculated by finding the intersection of intervals at each confidence level and integrating over the shared region.

Value

A list containing:

overlap_area Estimated area of overlap

total_area Total combined area

overlap_ratio Ratio of overlap to total area

max_shared_level Maximum confidence level where intervals overlap

See Also

curve_compare() for graphical comparison

plot_compare() for visualization

Examples

```
## Not run:
# Compare two study results
study1 <- curve_from_ratio(1.5, 1.1, 2.0)
study2 <- curve_from_ratio(1.3, 0.9, 1.8)

overlap <- curve_overlap(study1[[1]], study2[[1]])
print(overlap)

## End(Not run)
```

`curve_rev` *Reverse Engineer Consonance / Likelihood Functions Using the Point Estimate and Confidence Limits*

Description

Using the confidence limits and point estimates from a dataset, one can use these estimates to compute thousands of consonance intervals and graph the intervals to form a consonance, surprisal, and likelihood functions. The intervals are calculated from the approximated normal distribution, however, users should be cautious as this this function is currently designed for similar situations (involving ratios and normal approximations), nevertheless the function also works for means but should be used skeptically, as it can break down in many situations and give implausible numbers. Computations of likelihood functions for means is currently not supported.

Usage

```
curve_rev(
  point,
  LL = NULL,
  UL = NULL,
  se = NULL,
  conf.level = 0.95,
  type = "c",
  measure = "ratio",
  steps = 10000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

`point`	The point estimate from an analysis. Ex: 1.20
`LL`	The lower confidence limit from an analysis Ex: 1.0
`UL`	The upper confidence limit from an analysis Ex: 1.4
`se`	The standard error of the point estimate. Ex: 0.05
`conf.level`	Confidence level of the interval estimate.
`type`	Indicates whether the produced result should be a consonance function or a likelihood function. The default is "c" for consonance and likelihood can be set via "l".

measure	The type of data being used. If they involve mean differences, then the "mean" option should be used. If the data are ratios, then the "ratio" option should be used. "ratio" is currently the default option. Currently, this function is designed to be used with ratios and normal approximations rather than means.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

See Also

ggcurve()

curve_compare()

plot_compare()

Examples

```
## Not run:
# From a real published study. Point estimate of the result was hazard ratio of 1.61 and
# lower bound of the interval is 0.997 while upper bound of the interval is 2.59.
#
df <- curve_rev(point = 1.61, LL = 0.997, UL = 2.59, measure = "ratio")

## End(Not run)
```

---

curve_snowflake *Construct Consonance Functions from Snowflake Query Results*

Description

Connects to a Snowflake database and constructs consonance distributions from query results containing point estimates and confidence intervals.

Usage

```
curve_snowflake(
  conn,
  query,
  estimate_col = "estimate",
  lower_col = "lower",
```

```
  upper_col = "upper",
  conf.level = 0.95,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

conn	A DBI connection object to Snowflake database.
query	SQL query string that returns columns for estimate, lower, and upper bounds.
estimate_col	Name of the column containing point estimates. Default is "estimate".
lower_col	Name of the column containing lower bounds. Default is "lower".
upper_col	Name of the column containing upper bounds. Default is "upper".
conf.level	Confidence level of the input intervals. Default is 0.95.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes a summary table.

Details

This function requires the DBI and odbc packages to be installed. It connects to Snowflake, executes the provided query, and constructs consonance functions from the results.

Value

A list with class "concurve" containing the intervals dataframe, density dataframe, and optionally a summary table.

See Also

curve_snowflake_batch() for batch processing

curve_rev() for constructing curves from published intervals

export_for_powerbi() for exporting results

Examples

```
## Not run:
library(DBI)
library(odbc)

# Connect to Snowflake
conn <- dbConnect(odbc::odbc(),
  Driver = "Snowflake",
  Server = "your_account.snowflakecomputing.com",
  Database = "your_database",
  Schema = "your_schema",
  UID = "your_username",
  PWD = "your_password"
)

# Query with estimate and CI columns
```

```
query <- "SELECT estimate, lower_ci, upper_ci FROM results WHERE analysis_id = 1"
result <- curve_snowflake(conn, query,
  estimate_col = "estimate",
  lower_col = "lower_ci",
  upper_col = "upper_ci"
)

ggcurve(result[[1]], type = "c")

dbDisconnect(conn)

## End(Not run)
```

---

curve_snowflake_batch *Batch Process Multiple Snowflake Queries for Consonance Functions*

Description

Executes multiple queries against a Snowflake database and constructs consonance distributions for each result set.

Usage

```
curve_snowflake_batch(
  conn,
  queries,
  estimate_col = "estimate",
  lower_col = "lower",
  upper_col = "upper",
  conf.level = 0.95,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

conn	A DBI connection object to Snowflake database.
queries	A named list of SQL query strings.
estimate_col	Name of the column containing point estimates.
lower_col	Name of the column containing lower bounds.
upper_col	Name of the column containing upper bounds.
conf.level	Confidence level of the input intervals. Default is 0.95.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation.
table	Logical. If TRUE (default), includes summary tables.

Value

A named list of concurve objects, one for each query.

See Also

curve_snowflake() for single query processing

plot_compare() for comparing results

Examples

```
## Not run:
queries <- list(
  "Model A" = "SELECT estimate, lower_ci, upper_ci FROM results WHERE model = 'A'",
  "Model B" = "SELECT estimate, lower_ci, upper_ci FROM results WHERE model = 'B'"
)

results <- curve_snowflake_batch(conn, queries,
  estimate_col = "estimate",
  lower_col = "lower_ci",
  upper_col = "upper_ci"
)

# Plot comparison
plot_compare(results[["Model A"]][[1]], results[["Model B"]][[1]])

## End(Not run)
```

---

curve_summary *Generate Summary Statistics for Consonance Objects*

Description

Produces a summary of key statistics from a consonance function.

Usage

```
curve_summary(
  data,
  levels = c(0.5, 0.9, 0.95, 0.99),
  null_value = NULL,
  digits = 4
)
```

Arguments

data	A concurve object or intervals data frame.
levels	Confidence levels to include in summary. Default is c(0.50, 0.90, 0.95, 0.99).
null_value	Reference value for compatibility assessment. Default is 0 for differences, 1 for ratios.
digits	Number of decimal places in output. Default is 4.

Details

This function provides a summary of a consonance function including:

- Point estimate (value at maximum consonance)
- Confidence intervals at specified levels
- P-value and S-value at the null hypothesis
- Interval width as a measure of precision

Value

A data frame with summary statistics.

See Also

`curve_table()` for formatted interval tables

Examples

```
## Not run:
model <- lm(mpg ~ wt, data = mtcars)
result <- curve_gen(model, "wt")

# Get summary
curve_summary(result[[1]])

# Custom levels
curve_summary(result[[1]], levels = c(0.80, 0.95))

## End(Not run)
```

`curve_surv` *Consonance Functions For Survival Data*

Description

Computes thousands of consonance (confidence) intervals for the chosen parameter in the Cox model computed by the 'survival' package and places the interval limits for each interval level into a data frame along with the corresponding p-value and s-value.

Usage

```
curve_surv(
  data,
  x,
  steps = 10000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

data	Object where the Cox model is stored, typically a list produced by the 'survival' package.
x	Predictor of interest within the survival model for which the consonance intervals should be computed.
steps	Indicates how many consonance intervals are to be calculated at various levels. For example, setting this to 100 will produce 100 consonance intervals from 0 to 100. Setting this to 10000 will produce more consonance levels. By default, it is set to 1000. Increasing the number substantially is not recommended as it will take longer to produce all the intervals and store them into a dataframe.
cores	Select the number of cores to use in order to compute the intervals The default is 1 core.
table	Indicates whether or not a table output with some relevant statistics should be generated. The default is TRUE and generates a table which is included in the list object.

Value

A list with 3 items where the dataframe of values is in the first object, the values needed to calculate the density function in the second, and the table for the values in the third if table = TRUE.

Examples

```
## Not run:
library(carData)
Rossi[1:5, 1:10]
library(survival)

mod.allison <- coxph(Surv(week, arrest) ~ fin + age + race + wexp + mar + paro + prio,
  data = Rossi
)
mod.allison

z <- curve_surv(mod.allison, "prio")

## End(Not run)
```

---

curve_table *Produce Tables For concurve Functions*

Description

Produces publication-ready tables with relevant statistics of interest for functions produced from the concurve package.

Usage

```
curve_table(data, levels, type = "c", format = "data.frame")
```

Arguments

data	Dataframe from a concurve function to produce a table for
levels	Levels of the consonance intervals or likelihood intervals that should be included in the table.
type	Indicates whether the table is for a consonance function or likelihood function. The default is set to "c" for consonance and can be switched to "l" for likelihood.
format	The format of the tables. The options include "data.frame" which is the default, "docx" (which creates a table for a word document), "pptx" (which creates a table for powerpoint), "latex", (which creates a table for a TeX document), and "image", which produces an image of the table.

See Also

ggcurve()

curve_compare()

plot_compare()

Examples

```
## Not run:
library(concurve)

GroupA <- rnorm(500)
GroupB <- rnorm(500)

RandomData <- data.frame(GroupA, GroupB)

intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData, method = "default")

(z <- curve_table(intervalsdf[[1]], format = "data.frame"))
(z <- curve_table(intervalsdf[[1]], format = "latex"))
(z <- curve_table(intervalsdf[[1]], format = "image"))

## End(Not run)
```

---

curve_wrap *Construct Consonance Functions from Any CI-Producing Function*

Description

A generic wrapper that constructs consonance, surprisal, and likelihood functions from any function that produces confidence intervals. This is the most flexible approach for generating consonance curves from arbitrary statistical procedures.

Usage

```
curve_wrap(
  ci_func,
  steps = 1000,
  cores = getOption("mc.cores", 1L),
  table = TRUE
)
```

Arguments

ci_func	A function that takes a confidence level (0-1) as its first argument and returns a numeric vector of length 2 (lower, upper bounds). See examples for common patterns.
steps	Number of consonance levels to compute. Default is 1000.
cores	Number of cores for parallel computation. Default uses getOption("mc.cores", 1L).
table	Logical. If TRUE (default), includes a summary table of key intervals in the output.

Details

This function repeatedly calls ci_func at different confidence levels to construct the full consonance function. The ci_func argument must be a function that:

1. Takes a confidence level (numeric between 0 and 1) as its first argument
2. Returns a numeric vector of exactly length 2: c(lower_bound, upper_bound)

Common patterns for ci_func:

- Linear models: function(level) confint.default(model, parm = "x", level = level)
- GLMs (profile): function(level) confint(model, parm = "x", level = level)
- t-tests: function(level) t.test(x, conf.level = level)$conf.int
- Correlations: function(level) cor.test(x, y, conf.level = level)$conf.int
- Proportions: function(level) prop.test(x, n, conf.level = level)$conf.int
- Survival: function(level) exp(confint(coxph_model, level = level)["var",])

Value

A list with class "concurve" containing:

Intervals Dataframe Data frame with columns: lower.limit, upper.limit, intrvl.width, intrvl.level, cdf, pvalue, svalue

Intervals Density Data frame for plotting density functions

Intervals Table Summary table of key intervals (if table = TRUE)

See Also

ggcurve() for plotting

curve_gen() for model-specific consonance functions

curve_rev() for constructing curves from published intervals

curve_table() for interval summaries

Examples

```
## Not run:
# Example 1: Linear model
data(mtcars)
model <- lm(mpg ~ wt + hp, data = mtcars)
ci_func <- function(level) confint.default(model, parm = "wt", level = level)
result <- curve_wrap(ci_func)
ggcurve(result[[1]], type = "c")

# Example 2: t-test
x <- rnorm(30, mean = 5)
ci_func <- function(level) t.test(x, conf.level = level)$conf.int
result <- curve_wrap(ci_func)
ggcurve(result[[1]], type = "c", nullvalue = 0)

# Example 3: Correlation
x <- rnorm(50)
y <- x + rnorm(50)
ci_func <- function(level) cor.test(x, y, conf.level = level)$conf.int
result <- curve_wrap(ci_func)

# Example 4: GLM with profile likelihood CIs
model <- glm(am ~ wt, data = mtcars, family = binomial)
ci_func <- function(level) confint(model, parm = "wt", level = level)
result <- curve_wrap(ci_func)

# Example 5: Proportion test
ci_func <- function(level) prop.test(45, 100, conf.level = level)$conf.int
result <- curve_wrap(ci_func)

# Example 6: Custom function with additional arguments
my_ci <- function(level, data, method) {
  cor.test(data$x, data$y, method = method, conf.level = level)$conf.int
}
df <- data.frame(x = rnorm(50), y = rnorm(50))
ci_func <- function(level) my_ci(level, data = df, method = "spearman")
result <- curve_wrap(ci_func)

## End(Not run)
```

---

export_for_powerbi *Export Consonance Data for Power BI*

Description

Exports consonance function data in formats optimized for Power BI visualization and analysis.

Usage

```
export_for_powerbi(
  data,
  file,
  format = NULL,
```

```
  include_metadata = TRUE,
  pivot = FALSE
)
```

Arguments

data	A concurve object or intervals data frame.
file	Output file path. Supports .csv, .xlsx, or .json extensions.
format	Output format: "csv" (default), "excel", or "json". If NULL, inferred from file extension.
include_metadata	Logical. If TRUE, includes additional metadata columns useful for Power BI DAX measures. Default is TRUE.
pivot	Logical. If TRUE, creates a pivoted format with separate columns for lower and upper bounds at each confidence level. Default is FALSE.

Details

This function prepares consonance function data for import into Power BI. The exported data includes:

- Confidence levels and corresponding intervals
- P-values and S-values
- Interval widths for precision analysis

When `include_metadata = TRUE`, adds columns for:

- `ci_label`: Human-readable confidence level (e.g., "95\
- `significance`: Whether interval excludes null value

Value

Invisibly returns the exported data frame.

See Also

`curve_table()` for formatted interval tables

`curve_snowflake()` for Snowflake database integration

Examples

```
## Not run:
# Generate consonance data
model <- lm(mpg ~ wt, data = mtcars)
result <- curve_gen(model, "wt")

# Export to CSV for Power BI
export_for_powerbi(result[[1]], "consonance_data.csv")

# Export with metadata
export_for_powerbi(result[[1]], "consonance_data.csv", include_metadata = TRUE)

# Export to Excel
```

```
export_for_powerbi(result[[1]], "consonance_data.xlsx", format = "excel")

## End(Not run)
```

---

ggcurve *Plots Consonance, Surprisal, and Likelihood Functions*

Description

Takes the dataframe produced by the interval functions and plots the p-values/s-values, consonance (confidence) levels, and the interval estimates to produce a p-value/s-value function using ggplot2 graphics.

Usage

```
ggcurve(
  data,
  type = "c",
  measure = "default",
  levels = 0.95,
  nullvalue = NULL,
  position = "pyramid",
  title = "Consonance Function",
  subtitle = "The function displays intervals at every level.",
  xaxis = expression(theta == ~"Range of Values"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Levels for CI (%)",
  color = colorspace::darken("#009E73", 0.5),
  fill = "#239a98"
)
```

Arguments

data	The dataframe produced by one of the interval functions in which the intervals are stored.
type	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example ggcurve (type = "s") or ggcurve(type = "c"). Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
measure	Indicates whether the object has a log transformation or is normal/default. The default setting is "default". If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes and their measures such as risk ratios, hazard ratios, and odds ratios.
levels	Indicates which interval levels should be plotted on the function. By default it is set to 0.95 to plot the 95% interval on the consonance function, but more levels can be plotted by using the c() function for example, levels = c(0.50, 0.75, 0.95).

`nullvalue` Indicates whether the null value for the measure should be plotted. By default, it is set to NULL, meaning it will not be plotted as a vertical line. Changing this to a numerical vector will specify the region where a line should be plotted or an area that should be shaded. The input must be a numerical vector, for example c(-0.5, 0.5) or a single numerical vector such as 0 or 1.

`position` Determines the orientation of the P-value (consonance) function. By default, it is set to "pyramid", meaning the p-value function will stand right side up, like a pyramid. However, it can also be inverted via the option "inverted". This will also change the sequence of the y-axes to match the orientation.This can be set as such, ggcurve(type = "c", data = df, position = "inverted").

`title` A custom title for the graph. By default, it is set to "Consonance Function". In order to set a title, it must be in quotes. For example, ggcurve(type = "c", data = x, title = "Custom Title").

`subtitle` A custom subtitle for the graph. By default, it is set to "The function contains consonance/confidence intervals at every level and the P-values." In order to set a subtitle, it must be in quotes. For example, ggcurve(type = "c", data = x, subtitle = "Custom Subtitle").

`xaxis` A custom x-axis title for the graph. By default, it is set to "Range of Values. In order to set a x-axis title, it must be in quotes. For example, ggcurve(type = "c", data = x, xaxis = "Hazard Ratio").

`yaxis1` A custom y-axis title for the graph. By default, it is set to "Consonance Level". In order to set a y-axis title, it must be in quotes. For example, ggcurve(type = "c", data = x, yxis1= "Confidence Level").

`yaxis2` A custom y-axis title for the graph. By default, it is set to "Levels for CI". In order to set a y-axis title, it must be in quotes. For example, ggcurve(type = "c", data = x, yxis2= "Confidence Level").

`color` Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to color = "#555555". The inputs must be in quotes. For example, ggcurve(type = "c", data = x, color = "#333333").

`fill` Item that allows the user to choose the color of the ribbons in the graph. By default, it is set to fill = "#239a98". The inputs must be in quotes. For example, ggcurve(type = "c", data = x, fill = "#333333").

Value

A plot with intervals at every consonance level graphed with their corresponding p-values and compatibility levels.

See Also

`plot_compare()`

Examples

```
## Not run:
# Simulate random data

library(concurve)

GroupA <- rnorm(500)
GroupB <- rnorm(500)
```

```
RandomData <- data.frame(GroupA, GroupB)

intervalsdf <- suppressMessages(curve_mean(GroupA, GroupB, data = RandomData, method = "default"))
ggcurve(type = "c", intervalsdf[[1]], nullvalue = c(0))

## End(Not run)
```

`ggplot_likelihood` *Plot likelihood using ggplot2*

Description

Plot likelihood using ggplot2

Usage

```
ggplot_likelihood(
  x,
  parameter = NULL,
  type = c("likelihood", "deviance", "both"),
  ci_level = 0.95,
  n_points = 200,
  theme = "minimal"
)
```

Arguments

`x`	A `likelihood_function` object
`parameter`	Name of the parameter to plot. If `NULL`, uses the first parameter.
`type`	Type of plot: `"likelihood"`, `"deviance"`, or `"both"`
`ci_level`	Confidence level (default: 0.95)
`n_points`	Number of parameter values for profile likelihood evaluation
`theme`	ggplot2 theme to apply. Options: `"minimal"` (default), `"classic"`, `"bw"`, or any other ggplot2 theme object.

Details

Creates publication-quality plots using ggplot2 with subtitle showing the confidence interval. For `type = "both"`, requires the patchwork package for combining plots vertically. If patchwork is not installed, only the likelihood plot is returned with a message.

Value

A ggplot object (or patchwork composition for `type = "both"`)

See Also

`plot` for base graphics version, `plotly_likelihood` for interactive plots

logLik.likelihood_function

Extract log-likelihood from a likelihood function

Description

Extract log-likelihood from a likelihood function

Usage

```
## S3 method for class 'likelihood_function'
logLik(object, ...)
```

Arguments

object	A likelihood_function object
...	Additional arguments (unused)

Details

Returns the log-likelihood evaluated at the maximum likelihood estimates. The value includes attributes that make it compatible with information criteria calculations (AIC, BIC, etc.).

Value

An object of class logLik containing the log-likelihood at the MLE, with attributes df (number of parameters) and nobs (number of observations)

plot.likelihood_function

Plot likelihood function

Description

Plot a likelihood function showing relative likelihood or deviance, with optional confidence intervals and reference lines.

Usage

```
## S3 method for class 'likelihood_function'
plot(
  x,
  parameter = NULL,
  type = c("likelihood", "deviance", "both"),
  n_points = 200,
  interval = NULL,
  add_ci = TRUE,
  ci_level = 0.95,
  add_mle = TRUE,
```

```
  relative = TRUE,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  col = "black",
  lwd = 2,
  ...
)

## S3 method for class 'likelihood_function'
plot(
  x,
  parameter = NULL,
  type = c("likelihood", "deviance", "both"),
  n_points = 200,
  interval = NULL,
  add_ci = TRUE,
  ci_level = 0.95,
  add_mle = TRUE,
  relative = TRUE,
  main = NULL,
  xlab = NULL,
  ylab = NULL,
  col = "black",
  lwd = 2,
  ...
)
```

Arguments

x	A likelihood_function object
parameter	Name of the parameter to plot. If NULL, plots the first parameter and displays a message.
type	Type of plot: "likelihood" (relative likelihood), "deviance" (profile deviance), or "both" (two-panel plot)
n_points	Number of parameter values at which to evaluate the profile likelihood
interval	A length-2 numeric vector specifying the range of parameter values to plot. If NULL, automatically determined as ±5 standard errors from the MLE.
add_ci	Logical; if TRUE, adds confidence interval bounds to the plot
ci_level	Confidence level for the intervals (default: 0.95)
add_mle	Logical; if TRUE, adds a vertical line at the MLE
relative	Logical; if TRUE, plots relative likelihood; if FALSE, plots absolute likelihood. Only applies to likelihood plots.
main	Main title for the plot. If NULL, automatically generated.
xlab	X-axis label. If NULL, uses the parameter name.
ylab	Y-axis label. If NULL, automatically determined by plot type.
col	Color for the main likelihood/deviance curve
lwd	Line width for the main curve
...	Additional arguments passed to plot

Details

Creates publication-quality plots of likelihood and deviance functions. Includes optional confidence intervals and reference lines. For two-panel plots, the layout is automatically managed.

Value

Invisibly returns the profile likelihood data frame

See Also

`ggplot_likelihood` for ggplot2 version, `plotly_likelihood` for interactive plots

`plotly_likelihood` *Interactive likelihood plot*

Description

Interactive likelihood plot

Usage

```
plotly_likelihood(x, parameter = NULL, ci_level = 0.95, n_points = 200)
```

Arguments

`x`	A `likelihood_function` object
`parameter`	Name of the parameter to plot. If `NULL`, uses the first parameter.
`ci_level`	Confidence level (default: 0.95)
`n_points`	Number of parameter values for profile likelihood evaluation

Details

Creates an interactive plot using plotly, allowing users to hover over the likelihood curve to see exact values. Includes confidence interval bounds and MLE reference line.

Value

A plotly object with interactive hover tooltips

See Also

`plot` for base graphics version, `ggplot_likelihood` for ggplot2 version

plot_all_parameters *Plot likelihood for all parameters*

Description

Plot likelihood for all parameters

Usage

```
plot_all_parameters(
  x,
  type = c("likelihood", "deviance"),
  ci_level = 0.95,
  n_points = 100,
  ncol = NULL,
  ...
)
```

Arguments

x	A likelihood_function object
type	Type of plot: "likelihood" or "deviance"
ci_level	Confidence level for intervals (default: 0.95)
n_points	Number of parameter values for profile likelihood evaluation
ncol	Number of columns in the plot grid. If NULL, automatically determined as ceiling(sqrt(n_parameters))
...	Additional arguments passed to plot

Details

Creates a multi-panel plot with one panel per parameter. Useful for visualizing all likelihood functions simultaneously for comparison.

Value

Invisibly returns NULL

plot_ci_levels *Plot likelihood with multiple confidence levels*

Description

Plot likelihood with multiple confidence levels

Usage

```
plot_ci_levels(
  x,
  parameter = NULL,
  ci_levels = c(0.5, 0.68, 0.9, 0.95, 0.99),
  n_points = 200,
  colors = NULL,
  ...
)
```

Arguments

x	A likelihood_function object
parameter	Name of the parameter to plot. If NULL, uses the first parameter.
ci_levels	Numeric vector of confidence levels to display (default: 50%, 68%, 90%, 95%, 99%)
n_points	Number of parameter values for profile likelihood evaluation
colors	Character vector of colors for each confidence level. If NULL, automatically generated as a gradient from blue to red.
...	Additional arguments passed to plot

Details

Plots a single likelihood function with colored interval bands at multiple confidence levels, allowing visual comparison of interval widths and how confidence intervals change with level.

Value

Invisibly returns NULL

---

plot_compare *Graph and Compare Consonance, Surprisal, and Likelihood Functions*

Description

Compares the p-value/s-value, and likelihood functions using ggplot2 graphics.

Usage

```
plot_compare(
  data1,
  data2,
  type = "c",
  measure = "default",
  nullvalue = FALSE,
  position = "pyramid",
  title = "Interval Functions",
  subtitle = "The function displays intervals at every level.",
  xaxis = expression(theta == ~"Range of Values"),
```

```
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Levels for CI (%)",
  color1 = darken("#D55E00", 0.2),
  color2 = darken("#009E73", 0.2),
  fill1 = "#99c7c7",
  fill2 = "#d46c5b"
)
```

Arguments

data1	The first dataframe produced by one of the interval functions in which the intervals are stored.
data2	The second dataframe produced by one of the interval functions in which the intervals are stored.
type	Choose whether to plot a "consonance" function, a "surprisal" function or "likelihood". The default option is set to "c". The type must be set in quotes, for example plot_compare(type = "s") or plot_compare(type = "c"). Other options include "pd" for the consonance distribution function, and "cd" for the consonance density function, "l1" for relative likelihood, "l2" for log-likelihood, "l3" for likelihood and "d" for deviance function.
measure	Indicates whether the object has a log transformation or is normal/default. The default setting is "default". If the measure is set to "ratio", it will take logarithmically transformed values and convert them back to normal values in the dataframe. This is typically a setting used for binary outcomes and their measures such as risk ratios, hazard ratios, and odds ratios.
nullvalue	Indicates whether the null value for the measure should be plotted. By default, it is set to FALSE, meaning it will not be plotted as a vertical line. Changing this to TRUE, will plot a vertical line at 0 when the measure is set to " default" and a vertical line at 1 when the measure is set to "ratio". For example, plot_compare(type = "c", data = df, measure = "ratio", nullvalue = "present"). This feature is not yet available for surprisal functions.
position	Determines the orientation of the P-value (consonance) function. By default, it is set to "pyramid", meaning the p-value function will stand right side up, like a pyramid. However, it can also be inverted via the option "inverted". This will also change the sequence of the y-axes to match the orientation.This can be set as such, plot_compare(type = "c", data = df, position = "inverted").
title	A custom title for the graph. By default, it is set to "Consonance Function". In order to set a title, it must be in quotes. For example, plot_compare(type = "c", data = x, title = "Custom Title").
subtitle	A custom subtitle for the graph. By default, it is set to "The function contains consonance/confidence intervals at every level and the P-values." In order to set a subtitle, it must be in quotes. For example, plot_compare(type = "c", data = x, subtitle = "Custom Subtitle").
xaxis	A custom x-axis title for the graph. By default, it is set to "Range of Values. In order to set a x-axis title, it must be in quotes. For example, plot_compare(type = "c", data = x, xaxis = "Hazard Ratio").
yaxis1	A custom y-axis title for the graph. By default, it is set to "Consonance Level". In order to set a y-axis title, it must be in quotes. For example, ggcurve(type = "c", data = x, yxis1= "Confidence Level").

`yaxis2` A custom y-axis title for the graph. By default, it is set to "Levels for CI". In order to set a y-axis title, it must be in quotes. For example, ggcurve(type = "c", data = x, yxis2= "Confidence Level").

`color1` Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to darken("#D55E00", 0.4). The inputs must be in quotes.

`color2` Item that allows the user to choose the color of the points and the ribbons in the graph. By default, it is set to darken("#009E73", 0.4). The inputs must be in quotes. For example, plot_compare(type = "c", data = x, color = "#333333").

`fill1` Item that allows the user to choose the color of the ribbons in the graph for data1. By default, it is set to fill1 = "#239a98". The inputs must be in quotes. For example, plot_compare(type = "c", data = x, fill1 = "#333333").

`fill2` Item that allows the user to choose the color of the ribbons in the graph for data1. By default, it is set to fill2 = "#d46c5b". The inputs must be in quotes. For example, plot_compare(type = "c", data = x, fill2 = "#333333").

Value

A plot that compares two functions.

See Also

`ggcurve()`

`curve_compare()`

Examples

```
## Not run:
library(concurve)

GroupA <- rnorm(50)
GroupB <- rnorm(50)
RandomData <- data.frame(GroupA, GroupB)
intervalsdf <- curve_mean(GroupA, GroupB, data = RandomData)
GroupA2 <- rnorm(50)
GroupB2 <- rnorm(50)
RandomData2 <- data.frame(GroupA2, GroupB2)
model <- lm(GroupA2 ~ GroupB2, data = RandomData2)

randomframe <- curve_gen(model, "GroupB2")

plot_compare(intervalsdf[[1]], randomframe[[1]], type = "c")

## End(Not run)
```

plot_multi *Plot Multiple Consonance Functions*

Description

Creates a combined plot showing multiple consonance functions for comparison.

Overlays multiple consonance or surprisal functions on a single plot for direct visual comparison of effect estimates across groups, time periods, or studies.

Usage

```
plot_multi(
  ...,
  type = "c",
  measure = "default",
  nullvalue = NULL,
  position = "pyramid",
  title = "Comparison of Consonance Functions",
  subtitle = "Functions display intervals at every level.",
  xaxis = expression(theta == ~"Effect Size"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Confidence Level (%)",
  colors = NULL,
  alpha = 0.15,
  legend.position = "bottom"
)

plot_multi(
  ...,
  type = "c",
  measure = "default",
  nullvalue = NULL,
  position = "pyramid",
  title = "Comparison of Consonance Functions",
  subtitle = "Functions display intervals at every level.",
  xaxis = expression(theta == ~"Effect Size"),
  yaxis1 = expression(paste(italic(p), "-value")),
  yaxis2 = "Confidence Level (%)",
  colors = NULL,
  alpha = 0.15,
  legend.position = "bottom"
)
```

Arguments

...	Named concurve dataframes to compare. Names become legend labels. Alternatively, pass a single named list of dataframes.
type	Character. Type of function to plot: "c" for consonance (default), "s" for surprisal.

measure	Character. Scale type: "default" for linear, "ratio" for log scale (odds ratios, hazard ratios, etc.).
nullvalue	Numeric. Value(s) to mark with vertical reference line(s). Use single value (e.g., 0) or vector for range (e.g., c(-0.5, 0.5)).
position	Character. Orientation of consonance function: "pyramid" (default) or "inverted".
title	Character. Plot title.
subtitle	Character. Plot subtitle.
xaxis	Character or expression. X-axis label.
yaxis1	Character or expression. Primary y-axis label.
yaxis2	Character. Secondary y-axis label (for CI levels).
colors	Character vector. Colors for each curve. If NULL, uses colorblind-friendly palette.
alpha	Numeric. Transparency for ribbon fill (0-1). Default is 0.15.
legend.position	Character or numeric vector. Legend position. Default is "bottom".
labels	Character vector of labels for each curve. If NULL, uses names from input or generates sequential labels.

Details

This function overlays multiple consonance or surprisal functions on a single plot for easy comparison. Each curve is distinguished by color.

Curve names (passed as named arguments) are automatically used as legend labels. For example, `plot_multi("Study A" = curve1, "Study B" = curve2)` will use "Study A" and "Study B" as labels in the legend.

Value

A ggplot object.

A ggplot2 object (inheriting from ggplot).

See Also

`ggcurve()` for single curve plotting

`curve_overlap()` for quantifying overlap

`ggcurve()`, `plot_compare()`, `curve_compare()`

Examples

```
## Not run:
# Compare three analyses
result1 <- curve_from_ratio(1.5, 1.1, 2.0)
result2 <- curve_from_ratio(1.3, 0.9, 1.8)
result3 <- curve_from_ratio(1.8, 1.2, 2.5)

plot_multi(result1[[1]], result2[[1]], result3[[1]],
  labels = c("Study A", "Study B", "Study C"),
  nullvalue = 1,
  title = "Comparison of Odds Ratios"
```

```
)

## End(Not run)

## Not run:
# Compare two studies
study1 <- curve_from_se(point = 0.5, se = 0.2, df = 50)
study2 <- curve_from_se(point = 0.8, se = 0.25, df = 80)

plot_multi(
  "Study A" = study1[[1]],
  "Study B" = study2[[1]],
  nullvalue = 0,
  title = "Comparison of Treatment Effects"
)

# Using a list
curves_list <- list("Group 1" = study1[[1]], "Group 2" = study2[[1]])
plot_multi(curves_list, type = "s", nullvalue = 0)

## End(Not run)
```

`plot_profile_vs_wald` *Compare profile and Wald confidence intervals*

Description

Compare profile and Wald confidence intervals

Usage

```
plot_profile_vs_wald(x, parameter = NULL, ci_level = 0.95, n_points = 200, ...)
```

Arguments

`x`	A `likelihood_function` object
`parameter`	Name of the parameter to plot. If `NULL`, uses the first parameter.
`ci_level`	Confidence level (default: 0.95)
`n_points`	Number of parameter values for profile likelihood evaluation
`...`	Additional arguments passed to `plot`

Details

Plots the profile likelihood with both profile-based and Wald (normal approximation) confidence interval bounds overlaid for comparison. Differences between the two methods indicate departures from normality or asymmetry in the likelihood.

Value

Invisibly returns a list with elements `profile` and `wald` containing the respective confidence interval bounds

print.likelihood_function

Print a likelihood function object

Description

Print a likelihood function object

Usage

```
## S3 method for class 'likelihood_function'
print(x, ...)
```

Arguments

x	A likelihood_function object
...	Additional arguments (unused)

Details

Displays a summary of the likelihood function including family, link function, number of observations and parameters, convergence status, maximum likelihood estimates, and log-likelihood at the MLE.

Value

Invisibly returns x

print.summary.likelihood_function

Print a likelihood function summary

Description

Print a likelihood function summary

Usage

```
## S3 method for class 'summary.likelihood_function'
print(x, digits = 4, ...)
```

Arguments

x	A summary.likelihood_function object
digits	Number of digits for printing numeric values
...	Additional arguments (unused)

Details

Prints the coefficient table, dispersion, log-likelihood, AIC, and BIC.

Value

Invisibly returns `x`

`RobustMax` *Robust Max, an alternative to max() that doesn't throw a warning*

Description

Robust Max, an alternative to max() that doesn't throw a warning

Usage

```
RobustMax(x)
```

Arguments

`x`	A vector to find the maximum value of

Value

The max value from a vector

`RobustMin` *Robust Min, an alternative to max() that doesn't throw a warning*

Description

Robust Min, an alternative to max() that doesn't throw a warning

Usage

```
RobustMin(x)
```

Arguments

`x`	A vector find the minimum value of

Value

The minimum value from the vector

summary.likelihood_function *Summarize a likelihood function object*

Description

Summarize a likelihood function object

Usage

```
## S3 method for class 'likelihood_function'
summary(object, ...)
```

Arguments

object	A likelihood_function object
...	Additional arguments (unused)

Details

Computes the variance-covariance matrix from the information matrix at the MLE and provides a summary table with estimates, standard errors, and Wald test statistics.

Value

A summary.likelihood_function object containing coefficient estimates, standard errors, z-values, p-values, dispersion, log-likelihood, AIC, and BIC.

vcov.likelihood_function *Extract variance-covariance matrix from a likelihood function*

Description

Extract variance-covariance matrix from a likelihood function

Usage

```
## S3 method for class 'likelihood_function'
vcov(object, ...)
```

Arguments

object	A likelihood_function object
...	Additional arguments (unused)

Details

Computes the variance-covariance matrix as the inverse of the observed information matrix at the MLE. For Gaussian models, uses the unbiased estimator of variance (SSR/(n-p)) for compatibility with vcov.lm.

Value

A symmetric matrix of covariances between parameter estimates

Index

coef.likelihood_function, 3
confint.likelihood_function, 4
construct_likelihood, 5
curve_boot, 6
curve_compare, 7
curve_compare(), *20*, *22*, *28*, *41*, *43*
curve_corr, 8
curve_from_ratio, 9
curve_from_ratio(), *11*
curve_from_se, 10
curve_from_se(), *10*
curve_gen, 11
curve_gen(), *19*, *29*
curve_lik, 13
curve_lmer, 13
curve_mean, 15
curve_meta, 16
curve_model, 18
curve_overlap, 20
curve_overlap(), *43*
curve_rev, *10*, 21
curve_rev(), *10, 11*, *23*, *29*
curve_snowflake, 22
curve_snowflake(), *25*, *31*
curve_snowflake_batch, 24
curve_snowflake_batch(), *23*
curve_summary, 25
curve_surv, 26
curve_table, 27
curve_table(), *7*, *26*, *29*, *31*
curve_wrap, *18, 19*, 28
curve_wrap(), *19*

export_for_powerbi, 30
export_for_powerbi(), *23*

ggcurve, 32
ggcurve(), *7*, *19*, *22*, *28, 29*, *41*, *43*
ggplot_likelihood, 34, *37*

logLik.likelihood_function, 35

plot, *34*, *36–39*, *44*
plot.likelihood_function, 35
plot_all_parameters, 38
plot_ci_levels, 38
plot_compare, 39
plot_compare(), *7*, *20*, *22*, *25*, *28*, *33*, *43*
plot_multi, 42
plot_profile_vs_wald, 44
plotly_likelihood, *34*, *37*, 37
print.likelihood_function, 45
print.summary.likelihood_function, 45

RobustMax, 46
RobustMin, 46

summary.likelihood_function, 47

vcov.likelihood_function, 47
vcov.lm, *47*